

Министерство образования и науки Российской Федерации
Московский физико-технический институт (национальный
исследовательский университет)

Факультет радиотехники и компьютерных технологий
Кафедра

Выпускная квалификационная работа бакалавра по направлению 010900
«Прикладные математика и физика»

Изучение мохнатых существ с α Центавра

Студент 082 группы
Сидоров А. Б.

Научный руководитель
Иванов В. Г.

Долгопрудный
2026

Содержание

1. Введение	1
2. Постановка проблемы	3
2.1. Цели работы:	3
2.2. Задачи работы:	3
3. Обзор существующих решений	5
3.1. Верификация байт-кода динамически типизированных яп:	5
3.2. Верификация байт-кода статически типизированных яп:	6
4. Теоретическая часть	9
4.1. Устройство верификатора	9
4.1.1. Режимы верификации:	9
4.1.2. Схема работы:	10
4.2. ets opcodes	14
4.3. any opcodes	15
Благодарности	17

Глава 1

Введение

В настоящее время создается множество языков программирования, каждый из которых своей целью ставит удобство решения определенного класса задач. Большинство ныне существующих языков программирования относятся к платформо-независимым языкам. Главным их преимуществом является высокая переносимость кода, существует идиома описывающая данный вид поведения (WORA — Write Once, Run Anywhere). Примерами таких языков являются: ArkTS, C#, Dart, Java.

Высокая переносимость обеспечивается по средствам специальных сред исполнения, называемых виртуальными машинами, которые исполняют байт-код. Байт-код — промежуточный, платформенно-независимый код низкого уровня, получаемый при компиляции исходного текста программы. Он состоит из компактных инструкций, которые исполняются не напрямую процессором, а виртуальной машиной (например, JVM для Java), что позволяет запускать приложения на Windows, Linux, macOS и других платформах без перекомпиляции.

В связи со сказанным выше на первый план выходит проблема отслеживания корректности (здесь и далее под корректностью будем понимать соответствие архитектуры набора команд и семантике языка), сгенерированного байт-кода. Программы отслеживающие корректность называются верификаторами байт-кода. Важность изучения и разработки данных программы обуславливается тем, что исполняемые приложения могут отвечать за функционирование крайне чувствительных областей, таких как телекоммуникации, банковская сфера, медицинские услуги, где ошибка исполнения может привести к необратимому ущербу. Наличие верификатора позволяет обнаруживать ошибки до времени исполнения. Кроме того, верификатор оказывается полезен на этапе разработки языка программирования.

Существует несколько источников некорректного байт-кода:

1. Ошибка компилятора на этапе кодогенерации, Lowering (понижение), регистровой аллокации и т.д.
2. Повреждения пакета с исполняемым кодом во время его передачи по сети.

В данной работе фокус идет на повышение точности работы верификатора байт-кода путем расширения его типовой система и увеличения множества верифицируемых инструкций.

Глава 2

Постановка проблемы

2.1 Цели работы:

1. Расширить типовую систему верификатора для более точного отображения типовой системы виртуальной машины на типовую систему верификатора.
2. Разработать и внедрить алгоритмы проверок ряда инструкций, для расширения множества верифицируемых программ.

2.2 Задачи работы:

1. Изучить существующие решения.
2. Сравнить типовые системы виртуальной машины и верификатора, выявить неверифицируемые типы.
3. Расширить типовую системы и поддержать логику обработки данных типов.
4. Проанализировать архитектуру набора команд платформы и выявить множество неверифицируемых инструкций.
5. Реализовать и внедрить алгоритм верификации новых инструкций.
6. Протестировать каждый из предложенных алгоритмов.

Цели работы можно считать достигнутыми. Так как по ходу работы было реализовано порядка 10 обработок инструкций, что позволило выявить ошибки кодогенерации компилятора и улучшить гарантии платформы по безопасности. Как итог ahead-of-time(заранее) режим верификатора был добавлен в основную ветку OHOS

Глава 3

Обзор существующих решений

На данный момент большинство виртуальных машин обладают верификаторами, в качестве самых популярных примеров могут послужить JVM - виртуальная машина для исполнения байт-кода таких языков как: java, kotlin; .Net CLR - виртуальная машина для исполнения байт-кода C#, F#. Так же существуют верификаторы, которые не поставляются вместе с платформой, а являются сторонними библиотеками, примерами могут послужить BCEL Verifier для Java байт-кода и PEVerify, который проверяет на корректность IL-кода(C#). Рассмотрим некоторые из существующих решений и виды выполняемых проверок более подробно.

3.1 Верификация байт-кода динамически типизированных ЯП:

В динамически типизированных языках типы определяются в процессе исполнения: переменная может менять тип, возможно динамическое изменение прототипов. К тому же во многих языках есть конструкции, принимающие строки кода и выполняющие их уже на этапе runtime. В связи с этим список возможных проверок ограничен.

1. Проверки формата байткода:

Целью данных проверок является тестирование на валидность инструкций, корректность их формата и правильность индексов, соответствующих методам, полям и типам.

2. Проверки графа управления

Во время выполнения данного типа проверок, программа анализируется на корректность инструкций условного перехода: целевой адрес не выходит за пределы текущего метода, в середину try-блока или в середину другой инструкции. Проверяется отсутствие циклов с некорректным состоянием регистров. Контро-

лируется, что все пути выполнения заканчиваются инструкциями обозначающими *return* или *throw*

3. Сохранение инварианта стека

Размер стека должен быть неотрицательным числом, при этом его глубина не должна привывать некоторого заданного значения, в точках схождения глубина стека должна быть одинаковой.

4. Проверка Инлайн кэшей:

Inline Cache — это механизм ускорения динамического доступа в JS. Но для безопасного их использования нам надо гарантировать, что при обращении к конкретному слоту он будет существовать и будет инициализирован, а так же, что данный слот будет соответствовать типу текущей инструкции.

Важно отметить, что как правило движки динамически типизированных яп в себе не имеют такого объекта как верификатор, а перечисленные проверки выполняются непосредственно на этапе выполнения

3.2 Верификация байт-кода статически типизированных яп:

В статически типизированных языках типы переменных, аргументов и возвращаемых значений известны на этапе компиляции. Это позволяет компилятору генерировать байткод с встроенными гарантиями типов. Но байткод может быть сгенерирован не только компилятором, поэтому верификатор остается неотъемлемой частью виртуальной машины. Однако теперь возмжно выполнения ряда новых проверок, в добавок к тем, что уже были перечислены ранее:

1. Проверки типовой безопасности регистров:

Байткод инспектируется на предмет того, что тип каждого регистра является примитивным или ссылочным. Верифицируется, что любой регист использует-ся после своей инициализации.

2. Проверки инструкций вызова:

Инструкции вызова проходят проверку на корректность объекта ресивера(в случае нестатического метода). На количество аргументов, на типы данных аргументов и тип возвращаемого значения. Верифицируется возможность доступа.

3. Проверка доступа обращения:

Именно данная верификация является одной из самых важных для безопасности. Как известно, существует 3 основных модификатора доступа: публичные,

защищенные и приватные, которые применимы, как к элементам класса, так и классам и целым модулям. Контроль того, что их семантика не нарушается на уровне байткода необходим.

Рассмотрим пару примеров промышленных решений

1. ART

ART - Android Runtime В контексте данной работы для нас наибольший интерес представляют верификаторы тех платформ, которые предназначены для исполнения кода на мобильных устройствах. Именно таким является верификатор ART(Android runtime). ART исполняет DEX-байткод, источником которого могут быть: сторонние приложения, сгенерированного кода, потенциально повреждённых или злонамеренных источников.

2. LLVM

LLVM - Low level virtual machine LLVM-верификация — это обладает несколько иной философией, чем ART: она не про безопасность пользователя, а про корректность как ПП(здесь и далее ПП - промежуточное представление) компилятора. ПП генерируется доверенным фронтендом, но оптимизации могут его сломать. Можно выделить 4 уровня верификации: уровень модуля, функций, базовых блоков и инструкций. Проверяются как локальные, так глобальные инварианты.

Глава 4

Теоретическая часть

Рассмотрим некоторые теоретические аспекты, которые требуются для выполнения задач данной работы.

4.1 Устройство верификатора

4.1.1 *Режимы верификации:*

Верификация класса может начинаться в различные моменты времени, в зависимости от целей которые преследуются:

1. **Верификация времени установки** Всё приложение, а соответственно его классы, проходят проверку на этапе установки на устройство. Данный подход хорош, когда мы хотим исключить довольно дорогостоящие проверки из исполнения, ускорить старт приложения, возможность агрессивной компиляции в машинный код, всё перечисленное положительно сказывается на исполнении. Однако хоть такой подход и является стандартом индустрии, ультимативным он не является, т.к. имеет ряд недостатков: в данном режиме отсутствует информация какие классы реально будут загружены, какие ветки будут выполнены, какие типы появятся. Но самой главной проблемой данного подхода является то, что он плохо масштабируется по количеству изменений. Рассмотрим пример: у нас было несколько файлов, они были верифицированы и откомпилированы в ОАТ файл с машинным кодом, после были внесены какие-то изменения в исходный код, а так ОАТ файлы являются цельными артефактами, то они чувствительны к малейшим изменениям, которые могут сломать метоинформацию, то нам приходится перекомпилировать и проводить верификацию, что является довольно энергоёмким процессом.
2. **Верификация времени линковки** При данном подходе класс будет верифицирован во время его подготовки к исполнению, которая может быть вызвана, например, созданием его экземпляра. Но т.к. во время верификации будут

рекурсивно загружены все зависимые классы, что может привести к деоптимизациям, связанным с девиртуализацией(например например Оптимизация иерархии класса(chc)).

3. **Верификация "на лету"** Данный подход является наиболее гибким, верификация будет вызвана только при первом вызове метода. Так же данный подход является более инкрементальным и изменения проведенные в конкретном методе приведут только к верификации данного метода или его класса, такой подход более подходящий для jit.

4.1.2 Схема работы:

Сначала рассмотрим устройство верификатора целевой платформы. Единицей верификации является метод. Для каждого верифицируемого метода создается экземпляр объекта задачи для верифицирования(Job). Процесс создания задачи состоит из разрешения типов входных параметров и возвращаемого типа, построения графа потока управления, а также создания верификаторного контекста, который в себе содержит контекст регистров, который является одним из главных используемых инструментов во время посимвольного исполнения. Разберем каждый из этапов

Разрешение идентификаторов

Разрешение типов

Проверка графа потока управления

Верификатор исследуемой виртуальной машины выполняет большинство проверок графа потока управления, которые были перечислены ранее: проверка на то, что адрес перехода - начало инструкции, проверка отсутствия переходов за пределы метода и т.д. При этом структура метода имеет свою специфику, соответствие которой должно проверяться верификатором: последовательность блоков кода, перемежающихся с обработчиками исключений, поддерживая даже вложенные конфигурации обработчиков исключений, когда внутренний обработчик находится внутри внешнего. Однако эта гибкость уравнивается строгим набором правил, регулирующих переходы потока управления.

- Переходы из обработчика в код: Безусловные переходы из обработчика исключений обратно в обычный сегмент кода, запрещены. Это предотвращает произвольное возобновление нормального выполнения обработчиками исключений, вместо этого направляя их либо на повторное возбуждение исключения, явное завершение метода, либо на тщательно определенный путь восстановления, который не входит в основной поток кода в произвольной точке.



Рис. 4.1. Пример некорректного перехода.

Абстрактная интерпретация

Абстрактная интерпретация представляет собой формальный метод статического анализа программ, предназначенный для аппроксимации их поведения с целью доказательства корректности определённых свойств без выполнения программы на конкретных входных данных.

Метод был предложен Patrick Cousot и Radhia Cousot в 1970. Абстрактная интерпретация позволяет построить корректное приближение семантики программы, охватывающее все возможные её исполнения.

В процессе данного анализа происходит работа не с конкретными значениями, а с типами. При этом верификаторная система типов может отличаться от системы типов среды исполнения, ниже приведен пример системы типов целевой платформы. При этом отношение прототипирования для классов, определяемых пользователем, осуществляется по средствам механизмов среды исполнения.

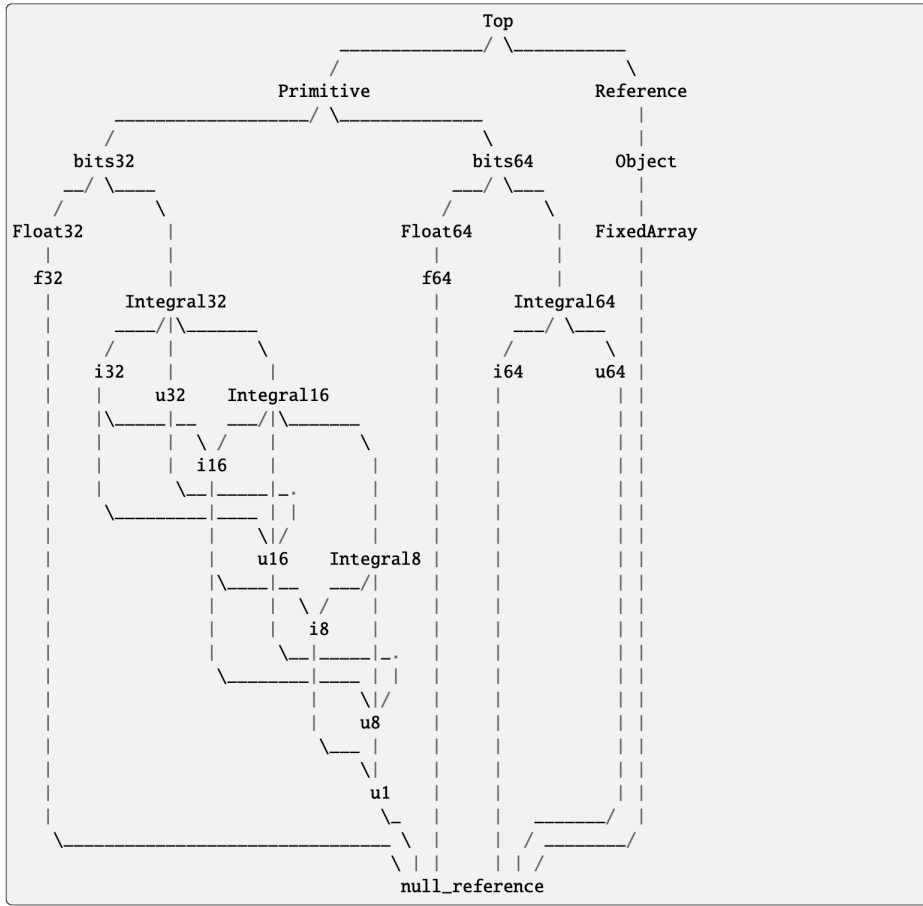


Рис. 4.2. Типовая система верификатора исследуемой платформы.

исследуемой и используя их свойства, строится некое приближение исполнения реальной программы, при этом данное приближение должно гарантировать корректность реально исполняемой программы. Данный алгоритм должен обладать следующими свойствами: *достоверность* - если верификатор успешно отработал, то ошибка времени исполнения не может возникнуть, *неполнота* - т.е. верификатор может отвергнуть валидный код. Разберем второе свойство более подробно. Кроме того анализ должен быть конечным

Пусть программа P обладает конкретной семантикой, которая описывает точное поведение программы при всех возможных входных данных. Конкретная семантика, как правило, оперирует бесконечными множествами состояний, что делает прямой анализ вычислительно неразрешимым, т.к. количество возможных состояний растет экспоненциально. Для решения данной проблемы вводится абстрактная семантика, которая: оперирует элементами абстрактного домена, является конечной, корректной аппроксимацией конкретной семантики. Между конкретной и абстрактной семантиками определяются функции:

- функция абстракции $\alpha : S_c \rightarrow S_a$;
- функция конкретизации $\gamma : S_a \rightarrow S_c$.

образующие галуасово соединение. По определению галуасово соединение это:

$$\begin{aligned} & [\forall c \in C \text{ и } \forall a \in A, \text{ где } A \text{ и } C \text{ некие абстрактный и конкретный домены}] \longmapsto \\ & \longmapsto [\alpha(c) \sqsubseteq_A a \iff c \sqsubseteq_C \gamma(c)] \end{aligned}$$

Выполнения данного условия является достаточным гарантирования корректности верификации. Рассмотрим, как данная теория отражена на практике.

Примерами *абстрагирования* могут послужить операции *соединения*(*join*) и *расширения*(*widennig*)

- Оператор *соединения* ($\sqcup$) представляет собой наименьшую верхнюю грань в абстрактном домене. Он заключается в объединении информации из различных путей выполнения программы для обеспечения наименее грубой аппроксимации, покрывающей несколько состояний. В алгоритмах абстрактной интерпретации *join* применяется в точках слияния графа потока управления. Программно это может выражаться в следующем: в одной ветке регистр v_1 имеет тип *String*, а по другой тип *int*. В итоге после слияния может произойти следующее:

$$String \sqcup Int = Object$$

, если типовая система верификатора(которая в общем и конкретно нас интересующем случае не равна типовой системе виртуальной машины) подразумевает, что *Object* — общий супертип(здесь и далее под супертипом подразумевается базовый класс). Или же возможно образования специального типа *объединение*(*union*)

$$String \sqcup Int = (String|Int)$$

- Для обеспечения завершения абстрактной интерпретации при анализе программ с циклами используется оператор *расширения*(∇). Расширение представляет собой специальную бинарную операцию над элементами абстрактного домена, ускоряющую достижение неподвижной точки путём ослабления точности аппроксимации. Применение расширения гарантирует завершение анализа за конечное число шагов, что является критически важным свойством для практических верификаторов байт-кода. Рассмотрим пример, когда используется данный подход:

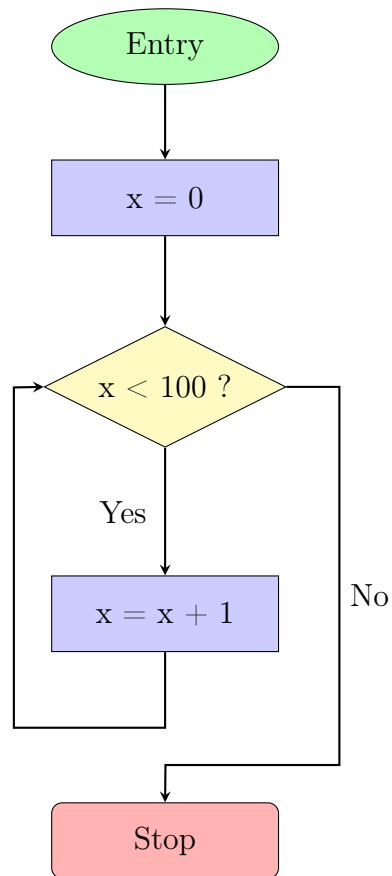


Рис. 4.3. Блок-схема цикла с условием для демонстрации операции *расширения*.

Используем интервальный домен:

$$x \in [l, r]$$

$$[l_1, r_1] \sqsubseteq [l_2, r_2] \Leftrightarrow l_2 \leq l_1 \wedge r_1 \leq r_2$$

На обратной дуге мы заменили возрастающую цепочку принимаемых x значений:

$$[0, 0] \sqsubseteq [0, 1] \sqsubseteq [0, 2] \sqsubseteq \dots$$

Данная цепочка может возрасть долго, поэтому домен состояний x будет определен, как $[0, +\infty]$

4.2 ets opcodes

Одной из отличительных особенностей целевой платформы от java-подобных платформ является поддержка объединения типов и наличие специального набора инструкций для эффективного взаимодействия с ними на уровне виртуальной машины. Под объединением типов в данной работе подразумевается ссылочный тип, который представляет из себя комбинацию нескольких типов.

```
1 class A { foo() { return 1 }}
2 class B { foo() { return 2 }}
3
4 function bar(v : A|B) {
5     v.foo()
6 }
```

Листинг 4.1. Пример кода с объединением типов

```
1 lda.obj a0
2 sta.obj v1
3 ets.call.name.short union.%%union_prop-A|B.foo:(union.%%union_prop-A
4 |B), v1
return.void
```

Листинг 4.2. Представление кода из Листинга 4.4 на уровне байткода

Компилятор генерирует синтетический класс, на этапе кодогенерации будет построена инструкция вызова метода данного класса, данная логика распространяется и на обращение к полям объединения. Важным является тот факт, что язык запрещает создание объекта объединения через оператор `new`. Таким образом на этапе исполнения в регистре будет находиться экземпляр конкретного класса, а из структуры инструкции можно однозначно получить имя метода или поля к которому идет обращение. Однако на этапе верификации отсутствует информации о значении регистра.

4.3 any opcodes

Также важной особенностью исследуемой платформы является поддержка взаимодействия с динамической средой. При этом данная особенность потенциально может послужить источником ошибок.

В виртуальной машине ArkTS взаимодействие с динамикой осуществляется по средствам `any` байткодов

```
1 // dynamic.js
2 class A { prop = 1}
3 function foo() { return new A() }
4
5 // static.ets
6 import { A, foo } from "./dynamic.js"
7 let a = foo()["prop"]
```

Листинг 4.3. Пример кода, взаимодействия с динамикой

```
1 lda.obj a0
2 sta.obj v1
3 any.ldbyname v1, "prop"
4 return.obj
```

Листинг 4.4. Пример кода, взаимодействия с динамикой

Благодарности

Благодарности идут тут.